AUDIO SPECTRUM ANALYZER

BY CHARBEL ABOU AFECH, KARLO MONTALLANA

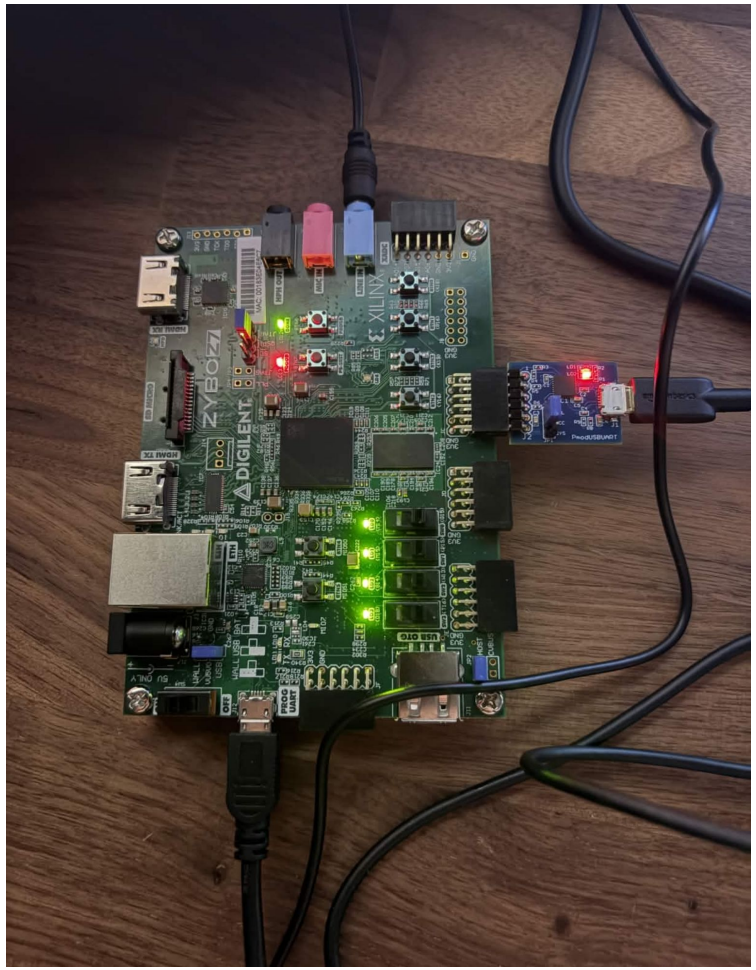
Project goal

What This project does



- Captures live audio from the Zybo Z7-10's onboard 3.5 mm line-in jack
- Computes a real-time 1024-point FFT entirely inside FPGA fabric — no CPU, no OS, no software DSP
- Streams frequency-domain magnitude bins over UART to a Python visualizer on a PC
- Renders a live 0–24 kHz bar-chart spectrum plot with no dropped frames
- Every design decision flows from one constraint: the signal path is pure synthesized logic

Hardware Platform

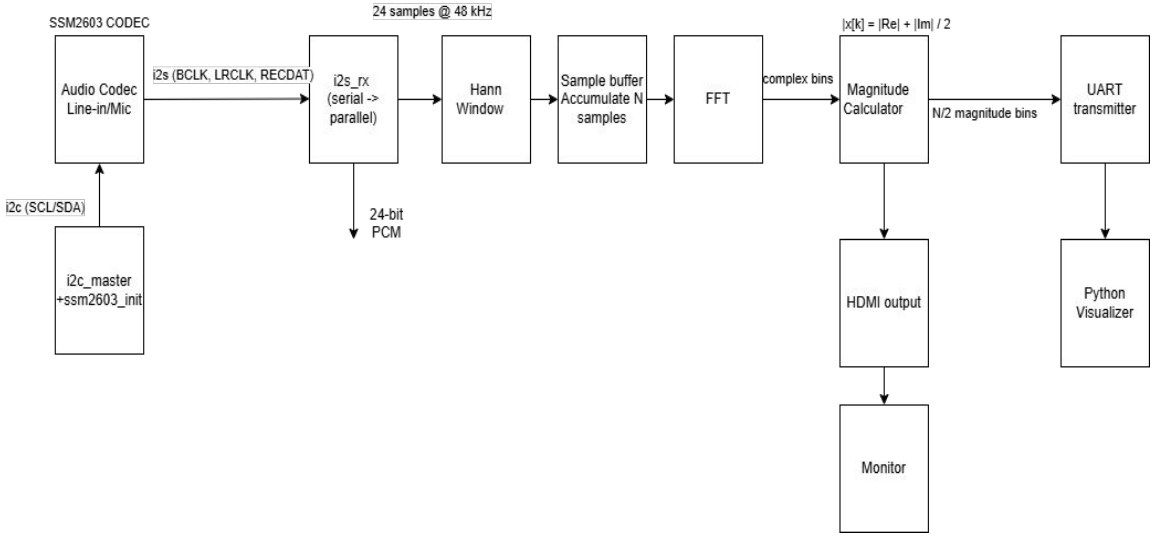


- Zybo Z7 board with SSM2603 audio codec.
- Analog source connected through line-in.
- On-board codec initialized through I2C.
- I2S stream captured inside FPGA logic.
- PMOD USB-UART bridge used for spectrum data into PC.
- 25 MHz system clock — single clock domain for the entire design

Key Clocks

- SYS_CLK = 125 MHz (board)
- MCLK = 12.5 MHz (125 MHz ÷ 10, generated in FPGA, fed to codec)
- BCLK = 1.536 MHz (codec-generated)
- LRCLK = 48 kHz (codec-generated)

System Architecture



- Codec Init: SSM2603 writes 9 registers over I²C at startup; rest of system waits
- Audio Capture: I2S samples BCLK/LRCLK/SDATA; valid pulse every 1/48,000 s
- Frame Assembly: FSM counts 1024 samples, pulses the frame to start, feeds FFT
- FFT + Magnitude: 1024 bins out, 25-bit magnitude each
- Buffer: Dual-bank ping-pong RAM; FFT writes one bank while UART reads the other
- UART: 21-state FSM sends 518-byte packet; 256 bins, 16-bit each, with header + checksum

VHDL module breakdown

Module	Role
top.vhd	Top-level: clock gen, MCLK, I²C tristate, frame FSM, LEDs
ssm2603_init.vhd	I²C config sequencer: 9-register startup FSM
i2c_master.vhd	I²C master, 100 kHz, start/stop/ACK
i2s_receiver.vhd	3-stage sync + I2S deserializer → 16-bit L/R samples
spectrum_uart_top.vhd	Main pipeline wrapper + 21-state UART packet FSM
spectrum_core.vhd	Connects fft_wrapper and magnitude_calc
fft.vhd	AXI4-Stream state machine around xfft_0 Xilinx IP
mag_calc.vhd	L1 magnitude: |Re| + |Im|, 25-bit, 1-cycle latency
buffer.vhd	Dual-bank ping-pong RAM, 1024 × 25-bit per bank
UART_transmitter.vhd	UART TX, 4-state FSM, parametric baud rate

FFT Physics & Frequency Resolution

Key Parameters

- N = 1024 points
- Fs = 48,000 Hz
- Bin spacing Δf = Fs / N = 46.875 Hz/bin — minimum resolvable frequency difference
- Nyquist limit = Fs / 2 = 24,000 Hz — hard ceiling; signals above this alia
- s
- One frame covers N / Fs = 21.33 ms of audio — this is also the UART deadline

$$\mathrm{x}[k] = \sum x[n] e^{\frac{-2\pi jkn}{N}}, for\ n = 0\,..\,N-1$$

$$\mathrm{x}[k] = |Re[k]| + |Im[k]|$$

$$Bin\ Spacing\ \Delta \mathrm{f} = {}^{F_s}/_{N}$$

FFT Physics & Frequency Resolution

Real Input Symmetry

- For real-valued input (imaginary = 0), bins 512–1023 are conjugate mirrors of bins 0–511
- Only first 512 bins are unique; project transmits bins 0–510 (0 to 23,906 Hz)

Aliasing

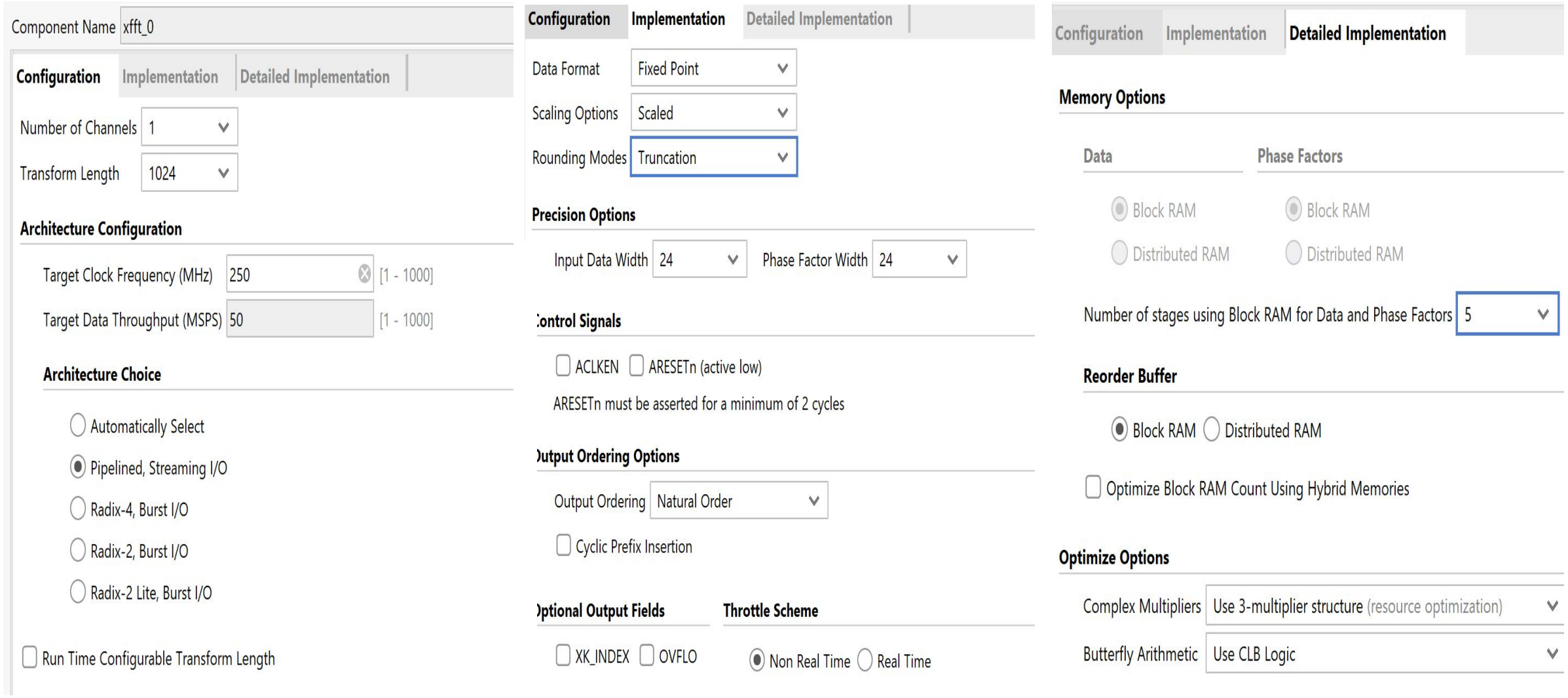
- Any signal above 24 kHz that passes the codec's anti-aliasing filter folds back
- Example: 40 kHz 2nd harmonic of a 20 kHz tone aliases to 48,000 − 40,000 = 8,000 Hz
- Visible as a secondary cluster at ~9 kHz when playing a 20 kHz tone — this is physics, not a bug

$$\mathrm{x}[k] = \sum x[n] e^{\frac{-2\pi jkn}{N}}, for\ n = 0\ ..N-1$$

$$\mathrm{x}[k] = |Re[k]| + |Im[k]|$$

$$Bin\ Spacing\ \Delta \mathrm{f} = {}^{F_s}/_{N}$$

Xilinx FFT IP Block



I2S Protocol & i2s receiver.vhd

How I2S Works

- Three wires: BCLK (bit clock), LRCLK (word select), SDATA (serial data)
- LRCLK low = left channel, LRCLK high = right channel
- Data is MSB-first; arrives one BCLK after the LRCLK edge (I2S protocol delay)
- Codec is master — FPGA reads signals passively

Clock Domain Crossing Problem

- BCLK (1.536 MHz) and LRCLK (48 kHz) are asynchronous to the 125 MHz FPGA clock
- Without synchronizers, metastability causes random bit errors

```
-- Synchronize codec signals into system clock domain
process(clk)
begin
    if rising_edge(clk) then
        bclk_sync  <= bclk_sync(1 downto 0) & bclk;
        lrclk_sync <= lrclk_sync(0) & lrclk;
        sdata_sync <= sdata_sync(0) & sdata;
    end if;
end process;

-- Rising edge of BCLK (3-stage: 2 for sync + 1 for edge detect)
bclk_rise <= '1' when bclk_sync(2 downto 1) = "01" else '0';
```

I2S Protocol & i2s receiver.vhd

i2s_receiver.vhd Solution

- BCLK: 3-stage synchronizer — 2 stages for metastability, 3rd stage for rising-edge detection (bclk_sync(2:1) = "01")
- LRCLK, SDATA: 2-stage synchronizer each
- On each BCLK rising edge: check if LRCLK changed (channel boundary) or shift data in
- Skip bit_cnt = 0 explicitly — implements the I2S one-clock delay
- Shift bits 1–16 into the shift register (16 bits per channel)
- On LRCLK falling edge: latch right channel, pulse valid

Top Level Usage

- Only left channel used: sign-extended from 16-bit to 24-bit signed for FFT input
- Right channel captured but discarded

```
            elsif bclk_rise = '1' then
                -- Track LRCLK transitions on BCLK rising edges
                lrclk_prev <= lrclk_sync(1);

                if lrclk_sync(1) /= lrclk_prev then
                    -- LRCLK edge detected: latch the completed channel
                    if lrclk_sync(1) = '1' then
                        -- Rising: left channel just finished
                        left_reg <= shift;
                    else
                        -- Falling: right channel just finished, pair is ready
                        right_reg <= shift;
                        valid_r   <= '1';
                    end if;
                    -- Reset for new channel; this BCLK is the I2S dummy bit
                    bit_cnt <= 0;
                    shift   <= (others => '0');
                else
                    -- Shift in data bits (skip bit_cnt 0 = I2S one-clock delay)
                    if bit_cnt >= 1 and bit_cnt <= DATA_WIDTH then
                        shift <= shift(DATA_WIDTH - 2 downto 0) & sdata_sync(1);
                    end if;
                    if bit_cnt < 63 then
                        bit_cnt <= bit_cnt + 1;
                    end if;
                end if;
            end if;
        end if;
    end process;

    left_data  <= left_reg;
    right_data <= right_reg;
    valid      <= valid_r;

end rtl;
```

Codec Initialization

Overview

- 4-state FSM: ST_DELAY → ST_SEND → ST_WAIT_I2C → ST_DONE
- Iterates through a 9-entry compile-time register table
- Uses i2c_master.vhd at 100 kHz for physical bus transactions
- SSM2603 I²C address: 0x1A (0b0011010)

Register	Value	Purpose
R15	0x1E00	Software reset — clears all codec state
100 ms delay		Codec PLL needs time to stabilize
R6	0x0C1A	Power: line-in ✓, ADC ✓, mic off, DAC off
R0	0x0017	Left line-in: 0 dB, unmuted
R1	0x0217	Right line-in: 0 dB, unmuted
R4	0x0800	INSEL=0 → line input
R5	0x0A00	ADC high-pass filter on
R7	0x0E42	I2S format, 16-bit, master mode
R8	0x1001	USB clock mode, 48 kHz
R9	0x1201	Activate digital audio interface

Magnitude Calculation

The Problem

- FFT output per bin: complex number (XK_RE, XK_IM), each 24-bit signed
- Need a single positive scalar for display

True Magnitude (not used)

- mag = sqrt($Re^2 + Im^2$)
- Requires multipliers (DSP48 slices) + CORDIC square root iteration
- Multi-cycle latency, significant resource cost

L1 Approximation (implemented)

- mag = |Re| + |Im|
- One line of combinational logic: ('0' & re_abs) + ('0' & im_abs)
- 25-bit result, 1 clock cycle registered latency
- 0 DSP48 slices used

Error Analysis

- Maximum error: $\times\sqrt{2}$ (~41% overestimate) when Re = Im (signal at 45° phase)
- Minimum error: 0% when either Re or Im is zero
- For a bar-chart spectrum display with ~200 px vertical resolution, 41% error = imperceptible
- No calibration requirement so L1 is the correct trade-off

Ping Pong Buffer

The Problem

- FFT writes 1024 magnitudes in ~21.33 ms
- UART reads 256 of them and transmits over ~16.58 ms
- Both happen concurrently — a single RAM would cause read/write collisions

Solution: Dual-Bank RAM

- Two independent 1024 × 25-bit RAMs: ram0 and ram1
- write_bank controls which bank the FFT writes into
- UART always reads from the opposite bank

Bank Swap Logic

- On mag_last (end of FFT frame): checks frame_available
  - If frame_available = 0 (UART idle): swap banks, set frame_available = 1
  - If frame_available = 1 (UART busy): drop the frame silently, increment overflow counter
- UART finishes → pulses frame_ack → clears frame_available → next swap allowed

Guarantee

- UART has exclusive read access to its bank for the entire packet transmission
- FFT is never stalled — it always has a bank to write into
- At most one frame per cycle is dropped (only when UART can't keep up)

UART Transmission & Timing

Bin Selection

- rd_send_idx runs 0–255; buffer address = rd_send_idx × BIN_STRIDE = rd_send_idx × 2
- Transmits bins 0, 2, 4, … 510 → covers 0 to 23,906 Hz at 93.75 Hz/bin effective resolution

Magnitude Scaling

- Raw 25-bit magnitude right-shifted by SCALE_SHIFT=4 (÷16), saturated to 16 bits
- Prevents saturation at 0xFFFF for loud audio; Python can rescale for display

Field	Size	Value
Magic Header	2 Bytes	0xAA, 0x55
Sequence number	1 Byte	Increments per packet
Bin Count	2 Bytes	0x0100 (256)
Magnitude Bins	512 Bytes	256 x 16-bit big-endian
XOR Checksum	1 Byte	XOR of all preceding bytes

Baud Rate Decision

Baud Rate	TX Time (518 B)	Frame Budget (21.33 ms)	Result
115,200	44.9 ms	2.1× over	drops every other frame
230,400	22.5 ms	1.06× over	still drops frames
312,500	16.58 ms	0.78× — fits	zero dropped frames

312,500 = 125,000,000 ÷ 400 → 0% baud rate error

Resource Utilization Report

Name	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	F8 Muxes (4400)	Block RAM Tile (60)	DSPs (80)	Bonded IOB (100)	BUFGCTRL (32)
N top	3942	5762	42	16	8.5	32	13	1
u_codec_init (ssm2603_init)	94	76	1	0	0	0	0	0
u_i2c (i2c_master)	68	42	1	0	0	0	0	0
u_i2s (i2s_receiver)	12	47	0	0	0	0	0	0
u_spectrum (spectrum_uart_top)	3811	5481	41	16	8.5	32	0	0
u_buffer (bin_buffer)	614	13	41	16	0	0	0	0
u_core (spectrum_core)	3049	5369	0	0	8.5	32	0	0
u_fft (fft_wrapper)	2917	5345	0	0	8.5	32	0	0
uut_fft (xfft_0)	2891	5313	0	0	8.5	32	0	0
u_mag (magnitude_calc)	132	24	0	0	0	0	0	0
u_uart_tx (uart_tx)	39	25	0	0	0	0	0	0

TIMING ANALYSIS

Design Timing Summary

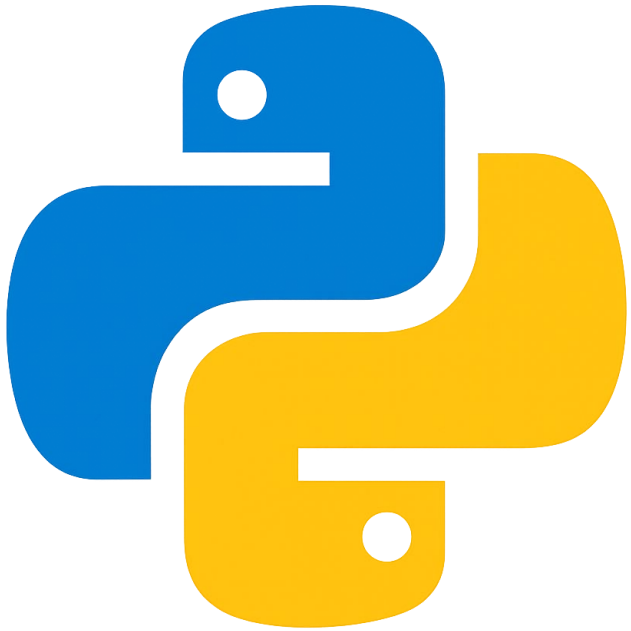
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	2.214 ns	Worst Hold Slack (WHS):	0.063 ns	Worst Pulse Width Slack (WPWS):	2.750 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	20887	Total Number of Endpoints:	20887	Total Number of Endpoints:	7509

All user specified timing constraints are met.

Python Visualizer (spectrum_analyzer.py)

Serial Reception

- PySerial at 312,500 baud, 1 ms timeout (non-blocking reads)
- Scans byte stream for magic header 0xAA 0x55
- Reads 516 bytes after header, validates XOR checksum



Frequency Reconstruction

- freq_hz = np.arange(256) * *BIN_STRIDE* * SAMPLE_RATE / NFFT
- = bins 0, 2, 4 … 510 mapped to 0, 93.75, 187.5 … 23,906 Hz

Display

- Bins below NOISE_FLOOR = 8 counts → set to 0 (prevents noise floor from filling display)
- matplotlib animated bar chart, ~10–20 FPS live update

Python Visualizer (spectrum_analyzer.py)

Per-Packet Terminal Output

- seq: sequence number (gaps indicate dropped frames)
- count: number of bins received (should always be 256)
- max: largest magnitude value in packet (0 = silence / no c
  connected)
- good / bad: cumulative packet counts
- dropped: inferred from sequence number gaps

Configs — Must Match FPGA Exactly

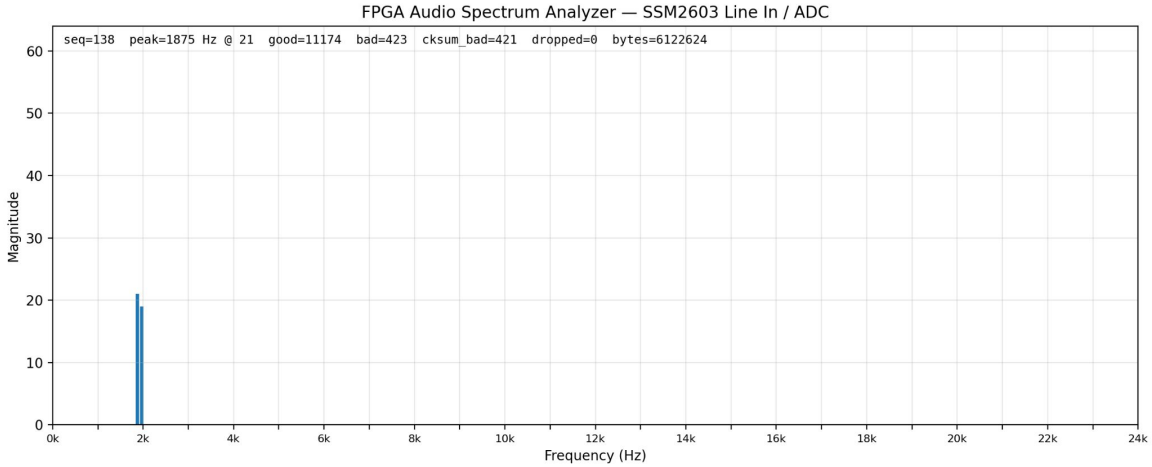
- BAUD_RATE = 312500
- EXPECTED_BINS = 256
- BIN_STRIDE = 2
- NFFT = 1024
- SAMPLE_RATE = 48000

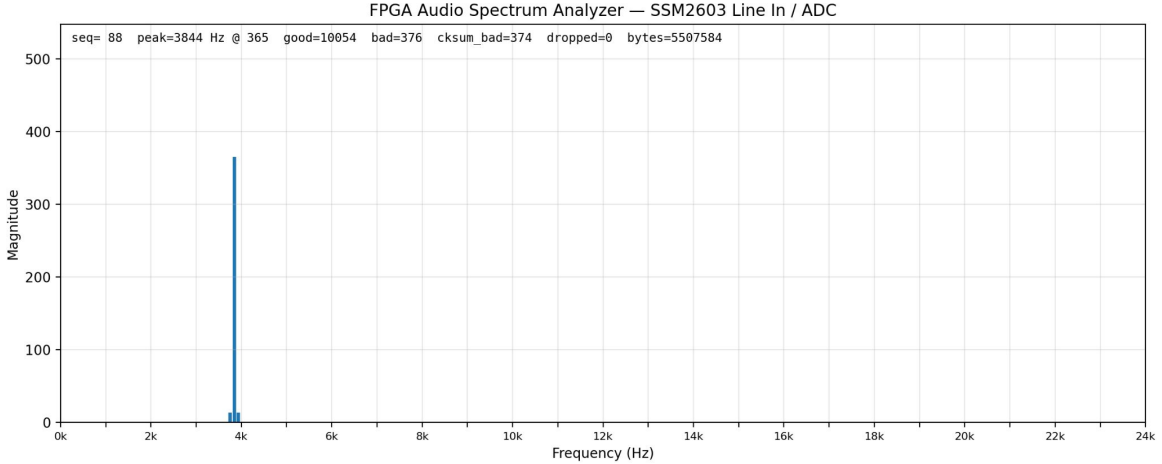
```
Opening serial port: COM3
Baud rate: 312500
Expected packet:
  header      = AA 55
  bin count   = 256
  packet size = 518 bytes

GOOD packet seq=189 count=256 max=  150 good=1 bad=0 dropped=n/a
GOOD packet seq=190 count=256 max=  118 good=2 bad=0 dropped=0
GOOD packet seq=191 count=256 max=  118 good=3 bad=0 dropped=0
GOOD packet seq=192 count=256 max=  124 good=4 bad=0 dropped=0
GOOD packet seq=193 count=256 max=   91 good=5 bad=0 dropped=0
GOOD packet seq=194 count=256 max=   52 good=6 bad=0 dropped=0
GOOD packet seq=195 count=256 max=   46 good=7 bad=0 dropped=0
GOOD packet seq=196 count=256 max=   66 good=8 bad=0 dropped=0
GOOD packet seq=197 count=256 max=   66 good=9 bad=0 dropped=0
GOOD packet seq=198 count=256 max=  113 good=10 bad=0 dropped=0
GOOD packet seq=199 count=256 max=  104 good=11 bad=0 dropped=0
GOOD packet seq=200 count=256 max=   79 good=12 bad=0 dropped=0
GOOD packet seq=201 count=256 max=  101 good=13 bad=0 dropped=0
GOOD packet seq=202 count=256 max=  116 good=14 bad=0 dropped=0
GOOD packet seq=203 count=256 max=  116 good=15 bad=0 dropped=0
GOOD packet seq=204 count=256 max=  133 good=16 bad=0 dropped=0
GOOD packet seq=205 count=256 max=   83 good=17 bad=0 dropped=0
```

Tone test results: low/mid band

Clear peaks were observed for 2 kHz and 4 kHz line-in test tones.





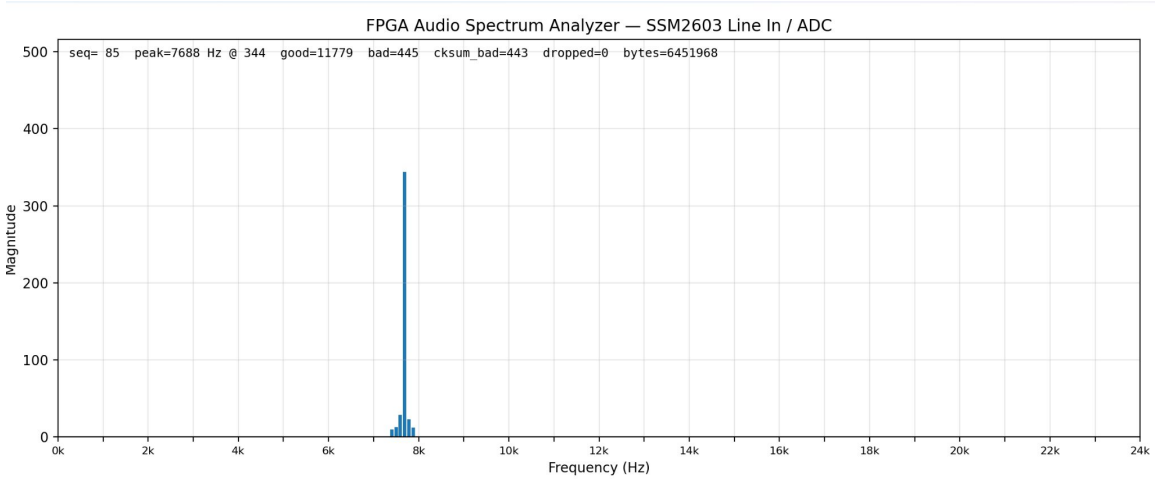
2 kHz input → displayed peak near 1.875 kHz

4 kHz input → displayed peak near 3.844 kHz

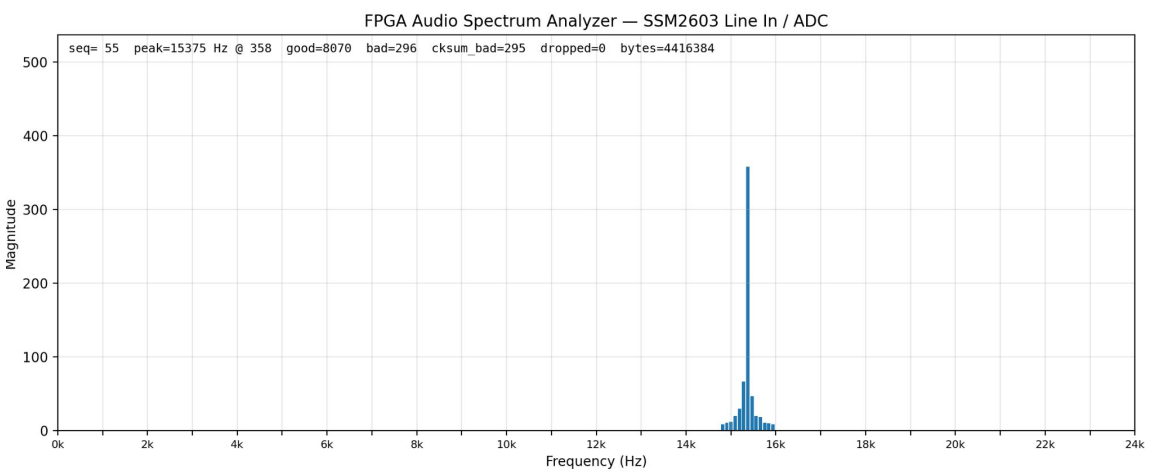
The small offsets are consistent with bin quantization and sample-rate / frequency-axis calibration effects.

Tone test results: higher band

Higher-frequency tones also produced dominant spectral peaks in the expected regions.



8 kHz input → displayed peak near 7.688 kHz



16 kHz input → displayed peak near 15.375 kHz

No dropped packets are shown in these captures; checksum errors are tracked by the visualizer for ongoing tuning.

What the fixed design demonstrates

Input tone	Displayed peak	Observation
2 kHz	**≈ 1.875 kHz**	Narrow low-frequency peak
4 kHz	**≈ 3.844 kHz**	Strong isolated peak
8 kHz	**≈ 7.688 kHz**	Stable high-mid peak
16 kHz	**≈ 15.375 kHz**	High-frequency peak visible

Interpretation

The hardware pipeline is successfully detecting dominant test-tone energy. Remaining improvement is mainly calibration/cleanup: frequency-axis mapping, checksum robustness, and integration polish.

Design Verification

- **Simulation:**

Testbench drove synthetic I2C slave ACKs and I2S waveforms at reduced clock (1 MHz) to verify codec init completes, MCLK toggles, and 50 stereo frames pass

- **Board LED checks:**

After programming the Zybo, LED0 blinking confirmed the 125 MHz clock and heartbeat; LED1/2/3 lighting up confirmed the FFT magnitude, UART TX, and packet completion stages were all active

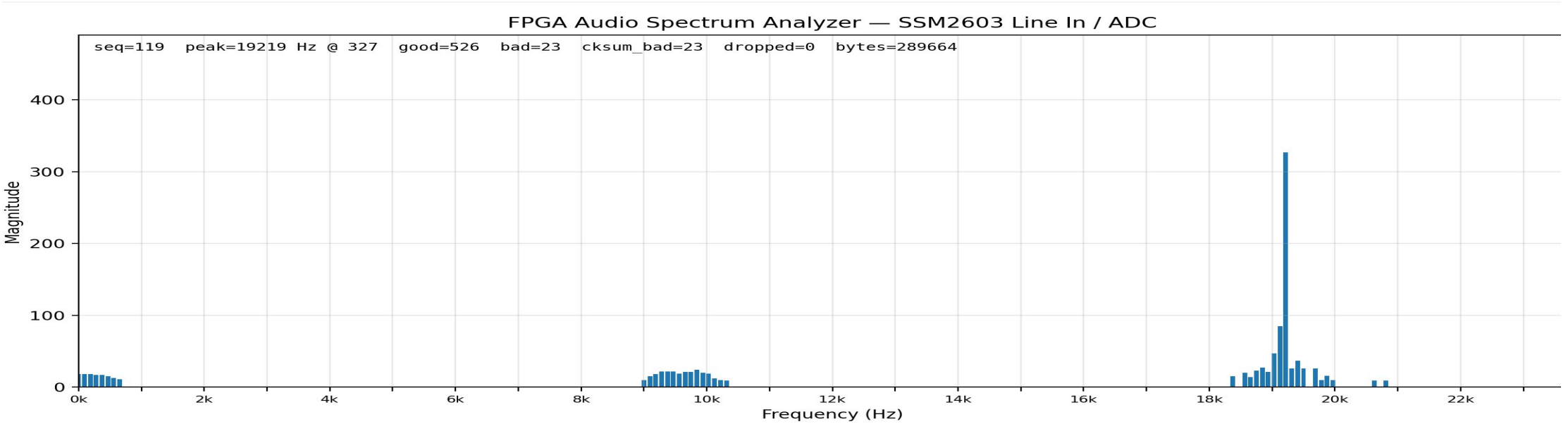
- **Python visualizer packet stats:**

Every packet showed count=256, dropped=0, and bad≤1 which confirms correct framing, zero missed FFT frames, and UART timing within 21.33 ms.

- **Tone tests:**

2 kHz, 4 kHz, 8 kHz, and 16 kHz tones each produced peaks at the nearest FFT bin just below the input frequency (1.875 kHz, 3.844 kHz, 7.688 kHz, 15.375 kHz respectively), consistent with 46.875 Hz/bin quantization rounding down; 20 kHz tone showed the correct

alias cluster at ~9 kHz, confirming Nyquist aliasing behavior



Challenges and fixes

The fixed version reflects several design corrections made during integration.

Codec + I2S timing	→	Synchronize BCLK/LRCLK/SDATA into the FPGA clock domain before deserializing.
FFT handshaking	→	Use a wrapper to drive valid/ready/tlast for exactly one 1024-sample frame.
UART bottleneck	→	Buffer spectrum bins so slow serial output does not stall or drop FFT results.
Visualization reliability	→	Track sequence, checksum, and dropped counters inside the Python plot.

Main takeaway: the analyzer needed both DSP correctness and careful data movement between clocked hardware stages.

Conclusion and next steps

What Was Achieved

- Real-time 0–24 kHz spectrum analyzer running entirely in Artix-7 FPGA fabric
- 1024-point FFT at 48 kHz → 46.875 Hz/bin frequency resolution
- Zero dropped UART frames at 312,500 baud with 4.75 ms margin
- Clean 10-module VHDL hierarchy, each independently verifiable
- Live Python visualization with noise floor filtering and scaled display
- All 6 major bugs identified and fixed through systematic debugging

Possible Future Work

- Window functions (Hann, Hamming) to reduce spectral leakage on pure tones
- Logarithmic (dB) magnitude scale for better dynamic range — human hearing is logarithmic
- Waterfall/spectrogram display: 2D plot of frequency vs time
- Stereo: transmit and display left and right channels independently
- Peak-hold and time-averaging in the Python visualizer
- Increase FFT size to 2048 for 23.4 Hz/bin resolution (requires more BRAM)